

NCSA — Network Compliance & Security Auditor

AI-augmented, vendor-agnostic configuration compliance for heterogeneous networks

Smart India Hackathon 2026 · Idea Submission

1. The Problem

Network devices are the primary gatekeepers of enterprise data and the most common point of misconfiguration. Organisations must align them with CIS Benchmarks, NIST SP 800-53, DISA STIGs and ISO/IEC 27001 — across firewalls, routers, switches and cloud security groups from dozens of vendors.

Two gaps make this unsolvable with today's tools:

Syntactic diversity. A "secure password" setting is written differently on every platform. `allowHttpMgmt=off` on one appliance, `no ip http server` on another, `<http/>` absent from an XML tree on a third — all the same control.

Scalability and adaptation. The estate is not static. White-box NOS, cloud-native security groups and newly acquired hardware appear continuously. Traditional parsers fail on structures they were never written for, and the industry's answer is a *product release* per new vendor.

The market is bifurcated: manual checklist auditing, or expensive vendor-locked suites that support a fixed device list and lack flexibility for mixed estates.

2. Proposed Solution

A compliance engine built on one principle:

Show what the configuration says, what the device actually does, and the gap between them — and say "I don't know" when it cannot tell.

Three layers deliver that:

layer	question answered
Declared state	Is <code>ssh_version</code> set to 2, as CIS requires?
Effective state	After resolving every object, group and zone, what does this rule actually permit?
The gap	Where do those two disagree?

Layer 3 is the product. On a real production firewall assessed during development, the configuration declared plaintext web management **disabled** — and the rule table simultaneously permitted TCP/80 inbound on seven WAN interfaces. Both statements were true. A checklist tool sees only layer 1.

Innovation and uniqueness

- Every finding carries proof** — source file, line number and the exact raw text, plus the framework citation it maps to. A reviewer can verify any result without re-running the tool.
- It refuses to guess.** Seven result states including `UNKNOWN` and `MANUAL_REVIEW`. A value assumed from a platform default is forbidden from producing a failure. Coverage is reported beside the score, always.

- 3. **It learns new vendors without a code deployment.** Unrecognised settings are queued, ranked and proposed to an administrator, whose approval becomes a parser — as *data*, never as executable code.
- 4. **Independent cross-validation.** Three separate mechanisms answer the same questions by different means; disagreement is surfaced as a reviewable signal rather than silently resolved.

3. What We Built — Complete Feature Inventory

3.1 Security Baseline Model (SBM)

The vendor-neutral vocabulary every device is translated into.

- **119 fields** in a flat dotted namespace (`management.ssh.version` , `crypto.weak_ciphers` , `firewall.default_action`)
- Field whitelist is load-bearing in three places: it constrains AI output at the decoder level, drives the matcher's candidate space, and defines the coverage denominator
- Every value is an **Observation** carrying state, source, confidence and evidence — never a bare value
- Four observation states: `OBSERVED` , `DEFAULT_ASSUMED` , `NOT_OBSERVED` , `UNPARSED`

3.2 Ingestion — six grammar readers

reader	formats	vendors
Indented hierarchy	CLI with indentation	Cisco IOS/IOS-XE, ASA, Arista, Aruba
Brace/semicolon	nested block syntax	Juniper Junos
Block-structured	<code>config / set / end</code>	Fortinet, SonicWall CLI
Key-value export	base64 appliance backup	SonicWall SonicOS <code>.exp</code>
XML	structured export	Junos <code>display xml</code> , PAN-OS
JSON	API/cloud output	AWS security groups, Azure NSG, SONiC
Host firewall	live system state	Windows Firewall, Linux iptables

Structured input is preferred over text wherever a device offers it. An XML export has already been parsed by the device that wrote it — there is no grammar left to guess, which eliminates an entire class of error rather than managing it.

3.3 Object graph and reference resolution

Configurations reference *names*, not values. `HR_NET` is not a subnet — it is a name resolving, possibly through nested groups, to a set of subnets.

- Recursive resolution with **cycle detection**
- A missing reference returns `UNRESOLVED` , **never an empty set** — an empty set silently satisfies "nothing is exposed"
- Resolution **path** is carried, not just the answer: `RDP_SERVICE → tcp/3389`
- On one real appliance: **841 objects, 935 relationships** reconstructed

Measured impact: naive line-matching produced **225 critical findings** on that device. After resolving zones and service groups: **7**, each verified by hand against raw device keys.

3.4 Compliance engine

- **81 controls** across three tiers (core / extended / category)
- **11 operators**: equals, in, not_in, contains_none, min_count, max_count, gte, lte, is_set, matches, worst-of
- **Seven result states** — PASS, FAIL, PARTIAL, NOT_APPLICABLE, UNKNOWN, MANUAL_REVIEW, ERROR
- **Scoped evaluation with worst-of aggregation** — a device with five management lines where one permits telnet is a FAIL, not a pass
- **Preconditions as data** — a control testing a sub-property of a disabled feature is NOT_APPLICABLE, not a failure. A device with SNMP switched off cannot fail an SNMPv3 authentication check.

3.5 Multi-framework mapping with provenance

5,376 catalogue entries ingested:

framework	entries	licence handling
NIST SP 800-53	1,196	public domain — full text
DISA STIG	553	public domain — full text
CIS Benchmarks	3,506	identifier only — cited by number
ISO/IEC 27001:2022	121	identifier only — clause number + title

Plus **4,462 CIS recommendation bodies** extracted from 89 benchmark PDFs for local mapping use only.

Labels are derived from published crosswalks, not similarity scores:

STIG V-223211 → CCI-001967 → NIST IA-3.1 → ISO A.5.x

3,836 CCI→NIST mappings and 220 NIST→ISO OLIR entries. A derived label can never present as stronger than the label it came from.

Licence discipline is enforced in the type system, not by convention. Copyrighted framework text cannot reach a report, a commit or a training set — export paths raise rather than leak.

3.6 Rule hygiene

Deterministic policy analysis — set containment in evaluation order. No ML, because a finding that says "delete this rule" must be either right or wrong.

- **Shadowed rules** — unreachable because an earlier rule matches the same traffic with a different action
- **Redundant rules** — fully covered by an earlier rule with the same action
- **Unused rules** — enabled, zero packet matches
- **Overly permissive** — any-source *and* any-destination allows
- **Disabled clutter** and **orphaned objects**

Live result on a production firewall: 227 unused, 62 disabled, 55 overly permissive, 26 redundant, 5 shadowed.

The most valuable single finding: an administrator had written a **deny** rule for a specific host beneath an allow-any rule on the same zone pair. The deny has no effect. They believe traffic is blocked. It is not.

Self-verification: containment findings are cross-checked against the device's own packet counters. A rule the analysis calls unreachable that *has* matched traffic proves our logic wrong — reported as a dispute rather than dropped. This caught two genuine bugs, including the discovery that one platform auto-sorts rules by specificity rather than by priority number.

3.7 Reachability analysis

Not "is there a rule mentioning SSH" but "**can this packet get from here to there**" — evaluated the way the device does, first match wins.

- Per-query verdict with the deciding rule named
- Unevaluable rules above the answer are **caveated**, never ignored
- No stated default policy → **undecidable**, never assumed deny
- Program-scoped host rules decline to answer port questions

3.8 Multi-device topology

- Interface and network extraction per platform
- Adjacency **inferred from shared subnets**, and labelled as inferred
- End-to-end path evaluation: every device must permit, or it is blocked
- A device with no policy model makes the path **undecidable — never permitted**

Limits are stated on every answer: no routing tables, no NAT modelling.

3.9 Change tracking over time

- Snapshot per assessment; compare against history
- **Separates device change from analysis change**. If a mapping improved between runs, findings move while the configuration is untouched — reporting that as "3 new violations" would be undetectable by the operator
- `UNKNOWN → PASS` is classified **coverage change**, not improvement
- Device identity matched on *any* shared strong identifier, because two exports of one appliance rarely expose the same ones

Measured on two exports of one real firewall, before and after hardening: score 36.1% → 60.0%, nine controls improved, zero regressions, 17 rules changed.

3.10 Recertification workflow

- Rule ownership register with named owner, justification and expiry
- Findings: `unowned`, `expired`, `expiring`, `stale_and_unowned`
- **Deletion candidates require ≥2 independent signals** — no traffic, covered by an earlier rule, unreachable, no owner. Any one alone is a bad reason to delete a firewall rule, and every candidate is labelled *for review*, *not automatic removal*.

3.11 Log correlation

Turns a caveated finding into a decidable one:

counter	logs	verdict
0	none in window	confirmed unused
0	entries present	counter was reset — our finding was wrong
N	none	unlogged — an audit-trail gap, itself a finding

On one device: **38 rules with millions of matches and no logging at all.**

3.12 Vendor-agnostic detection layer

Works with **no mapping pack at all** — the floor under a device nobody has described to us. Vocabulary is vendor-independent even when grammar is not: `telnet` is spelled `telnet` everywhere; `dh-group1` is 768-bit Diffie-Hellman on every platform.

Three rules keep it honest: **positive evidence only** (it may report "contains a weak cipher", never "lacks logging"), **negation aware** (`no ip http server` must not fire), and **confidence 0.8, labelled** so it is distinguishable from a parsed observation.

On a real appliance with no pack, it found a cleartext credential the vendor-specific pack itself had missed.

3.13 The learning loop — adaptation without redeployment

The problem statement's central requirement, implemented end to end.

```
unmapped settings → classify → queue by NAME → rank → propose
→ administrator approves → parser written as DATA → device re-assessed
```

- Deduplicated **by setting name**, not by record: 84,214 unmapped records on one device collapse to **841 distinct names**. One approval covers every instance, on every device of that platform.
- Proposals are ranked by a three-signal matcher (below)
- **Approval is gated**. The mapping is written, 35 hand-verified results are re-run, and if any changes the write is **reverted** and the approval refused with the specific expectation that broke
- A rising pass rate is flagged even when nothing breaks — poisoning always makes things look better
- Approvals are **hash-chained**; editing any past approval breaks every hash after it

Parsers are generated as data, never as code. Declarative mappings validated against the 119-field whitelist, with an allow-listed operation set. A malicious configuration cannot execute anything, because there is no execution path — a critical property when the input is attacker-influenced.

3.14 Field matching — measured, not assumed

Mapping an unknown vendor setting to a schema field, benchmarked on **233 labelled pairs holding out an entire vendor** (the only split that reproduces the real question):

method	top-1	top-5
lexical baseline	33.9%	54.1%
+ value-type prior	49.4%	58.4%
semantic embeddings	41.6%	59.2%
semantic + type prior	51.1%	63.9%
fused + type prior	45.5%	64.8%

The largest single gain came from the value, not the model. 214 of 400 real settings are integers, and only 8 of 119 schema fields are integer-typed — the candidate space collapses 15× before any model runs. The type prior is applied as a *multiplier*, never a *filter*, so a mismatch is penalised but recoverable.

A local instruction model re-ranks only the ambiguous cases, converting recall into precision at ~91% when the answer is in the top five. It is invoked selectively: on a large device, 63% of candidates need it, the rest do not.

3.15 Risk scoring

```
risk = base(severity) × exposure × confidence
```

Exposure is computed from the object graph, not assumed — the same control is CRITICAL where internet-reachable and MEDIUM where internal. Two properties are deliberate: **absence of a graph never discounts risk**, and **weak evidence scores down, not up**. UNKNOWN carries no risk score at all — a number there would imply we knew.

3.16 Remediation — lockout-safe

Every competitor emits a fix list. A naive fix list applied to a production firewall **locks the administrator out**.

- Ordered by phase: prepare → enable → harden → **disable last**
- Wrapped in the vendor's own rollback (`commit confirmed` , `reload in`)
- A step disabling the **only live management transport** is held back entirely, not merely warned about
- Controls with no written remediation are **listed, never invented**

3.17 Quality infrastructure

artifact	claim	scale
Golden corpus	correctness — a human read the config and cited the line	5 cases, 35 expectations
Behaviour snapshot	change detection	9 samples
Test suite	regression	288 tests
Matching benchmark	accuracy, repeatable	233 labelled pairs

Every golden expectation states *why*, citing the line that justifies it — an expectation nobody can re-check is indistinguishable from a snapshot of whatever the tool printed that day.

3.18 Cross-validation — three independent mechanisms

1. **Method consensus** — vendor pack vs vendor-agnostic detector
2. **Parser consensus** — our hand-written grammar vs an independent implementation of the same syntax
3. **Implementation consensus** — our analysis vs an external reference engine

Disagreement is never auto-resolved. In this project it went *both ways* within a single run: once our detector's hint was wrong, once our pack was blind and the detector was right. A rule suppressing either side would have hidden a real weakness.

3.19 REST API and integration surface

Seven endpoints: bulk assessment, results, remediation plan, training queue, approval, framework catalogue, platform list.

Three invariants are enforced **in the schema**, so a front end cannot undo them: score never travels without coverage, UNKNOWN carries no risk object, and every failure carries evidence.

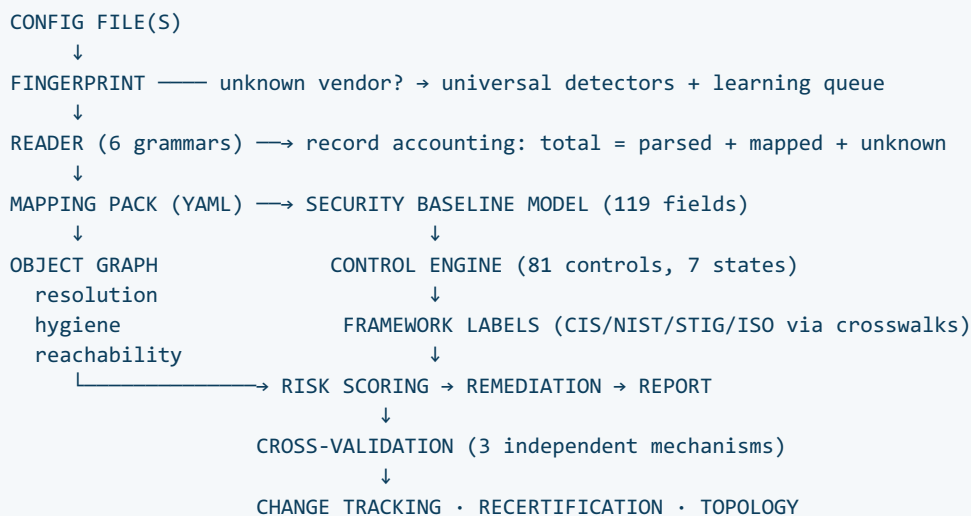
4. Technical Approach

Tech stack

layer	technology
Core engine	Python 3.11 , Pydantic (typed models, validation at boundaries)
Configuration format	YAML — vendor packs and controls are data, not code
API	FastAPI + Uvicorn, OpenAPI schema published
Retrieval	TF-IDF + BM25 lexical, local embedding model, reciprocal-rank fusion
AI tier	Local instruction model — runs entirely on-premises, no data leaves
Structured parsing	expat streaming XML, JSONPath, custom lexers
Reporting	HTML → PDF rendering
Frontend	React (design-led) against the published API contract
Testing	pytest — 288 tests, golden corpus, behaviour snapshots

~14,200 lines of engine code, ~2,800 lines of tests.

Architecture



Methodology

- **Data-driven vendor support.** Adding a vendor is a YAML file. No code change, no redeployment. 7 packs, 253 mappings today.
- **Evidence-first.** No finding exists without a file, line and raw text.
- **Measure, don't assume.** Every accuracy claim cites a repeatable benchmark and a date.
- **Fail loudly.** The tool refuses input it cannot genuinely read — an encrypted backup is declined with a reason rather than parsed into noise.

5. Feasibility and Viability

Feasibility — demonstrated, not projected

- Working engine validated against **real production devices**: two firewall appliance exports, a real ASA firewall, and a live host firewall with 703 rules
- Full framework catalogues ingested and mapped
- 288 automated tests; 35 hand-verified compliance outcomes
- Runs entirely on a laptop — no cloud dependency, no data egress

Risks and mitigations

risk	mitigation
AI produces a confident wrong mapping	Human approval gate; regression suite blocks any change that breaks a verified result; measured accuracy published rather than claimed
Untrusted config influences our logic	Parsers generated as data , validated against a fixed whitelist. No execution path exists. Injection pre-scan quarantines suspicious content before any model sees it
Copyrighted framework text leaking into output	Licence enforced in the type system; export paths raise rather than leak; local-only content excluded from version control
Coverage overstated to look good	Score is reported only over <i>decided</i> controls, always beside an assessed-coverage figure. <code>UNKNOWN</code> is a first-class state
A vendor we have never seen	Universal detectors give an immediate floor; the learning loop reaches a working parser in minutes, not a product release
Platform semantics differ from our assumptions	Cross-validation surfaces it — this is how we discovered one platform does not evaluate rules in priority order

Honest limitations

Stated in the codebase, not hidden: adjacency is inferred from shared subnets; routing tables and NAT are not modelled; multi-device analysis is policy composition, not full data-plane simulation.

6. Impact and Benefits

Operational

- **Minutes instead of days** for a multi-vendor audit that is currently manual
- **Any vendor in ~15 minutes** — including ones nobody has described to us — versus waiting a release cycle from a commercial suite
- **Actionable, safe remediation**: ordered CLI steps with rollback that will not lock an administrator out of a production device
- **Continuous, not one-shot**: change tracking turns an audit into monitoring

Security

- Finds the failures checklist tools structurally cannot: a deny rule with no effect, a control that passes while the device is exposed, an unlogged permit rule that breaks the audit trail

- **227 dead rules and 55 overly-permissive rules** surfaced on a single real firewall — attack surface nobody knew was there

Economic

- Removes vendor lock-in: no per-device licensing, no fixed support matrix
- Runs on commodity hardware; **no data leaves the organisation**, which makes it viable for regulated and air-gapped environments
- Open, data-driven architecture — an in-house team can add a vendor themselves

Trust

The differentiator a compliance officer cares about: **every number is defensible**. Each finding cites its evidence and its framework provenance, coverage is never overstated, and the tool says "I don't know" rather than guessing. An auditor who overstates coverage is a liability; this is built to be the opposite.

7. Research and References

Compliance frameworks implemented

- NIST SP 800-53 Rev 5 — Security and Privacy Controls (OSCAL catalogue)
- NIST OLIR — SP 800-53 to ISO/IEC 27001 crosswalk
- DISA Security Technical Implementation Guides (XCCDF), incl. Network Device Management STIGs
- DISA Control Correlation Identifier (CCI) list — CCI → 800-53 mapping
- CIS Benchmarks — 89 benchmark documents across 20+ vendor families
- ISO/IEC 27001:2022 Annex A

Vendor hardening documentation

- Cisco *Guide to Harden Cisco IOS Devices*
- Juniper Junos security policy references
- Fortinet FortiOS administration and CLI references
- Per-vendor CLI grammar and configuration references

Security engineering references

- MITRE ATLAS — adversarial threat landscape for AI systems (prompt-injection countermeasures)
- NIST AI Risk Management Framework
- OpenConfig / YANG — vendor-neutral network configuration modelling

Standards for output

- SCAP / XCCDF / OVAL — machine-readable compliance content
- Framework citation by identifier where redistribution is restricted

NCSA — every finding carries its evidence, every number carries its coverage, and the tool says "I don't know" rather than guessing.